CVT Unit 2 — Answer Sheet (Grouped by Topic)

Source: UNIT 2 updated.pdf | Questions: [QA] Unit2_QB - Part A & B.md

---

TOPIC GROUP 1: Points & Patches

[Part A | Q1 | 5 Marks] Define digital image formation.

Answer: A digital image is formed by capturing light reflected from a scene through a sensor (camera/CCD/CMOS) and converting the continuous intensity values into a 2D discrete grid of pixels. The image I(x, y) represents the intensity at spatial coordinates (x, y). The digitization process involves two steps:

- **Sampling** — dividing the spatial domain into a finite grid (determines spatial resolution).
- **Quantization** — mapping continuous intensity to discrete grey levels (e.g., 0–255 for 8-bit).

Each pixel stores a grayscale intensity or colour (RGB) value. The result is a rectangular array of numbers representing the scene.

[Part A | Q2 | 5 Marks] What is a pixel? What is a patch?

Answer:

- **Pixel** (picture element): The smallest addressable unit in a digital image, located at grid coordinates (x, y), storing a single intensity or colour value. It is the fundamental building block of a digital image.
- **Patch**: A small rectangular neighbourhood of pixels centred at a keypoint, typically 8×8, 16×16, or 32×32 pixels in size. A patch captures local texture, gradient orientation, and appearance around a feature point. Patches are used in template matching, feature description (e.g., SIFT uses a 16×16 patch), and training convolutional filters.

[Part A | Q3 | 5 Marks] Differentiate between point features and patch features.

Answer:

Aspect	Point Features	Patch Features
Definition	A single pixel location identified as a keypoint (corner, blob)	A region of pixels surrounding a detected keypoint
Information	Encodes only location (x, y)	Encodes appearance, texture, gradient distribution
Representation	Single coordinate pair	Multi-dimensional descriptor vector
Example	Harris corner, FAST keypoint	SIFT 128-dim descriptor, SURF 64-dim descriptor

Aspect	Point Features	Patch Features
Usage	Detection stage (where?)	Description & matching stage (what?)

Point features detect *where* to look; patch features describe *what* is there and enable matching across images.

[Part A | Q4 | 5 Marks] What is neighborhood in image processing?

Answer: A neighbourhood of pixel P is the set of pixels immediately surrounding P in the image grid. Two standard neighbourhoods are:

- **$N_4(P)$ — 4-neighbourhood**: The 4 pixels sharing an edge with P (up, down, left, right). Used in 4-connectivity.
- **$N_8(P)$ — 8-neighbourhood**: All 8 surrounding pixels (4 edge-sharing + 4 diagonal). Used in 8-connectivity.
- **Diagonal neighbours $N_D(P)$**: The 4 purely diagonal neighbours.

Neighbourhoods are fundamental to connected component analysis, morphological operations, edge detection filters (Sobel, Prewitt), and iterative image processing algorithms.

[Part A | Q5 | 5 Marks] Define 4-adjacency and 8-adjacency.

Answer:

- **4-adjacency**: Two foreground pixels P and Q are 4-adjacent if Q belongs to $N_4(P)$ — i.e., they are horizontally or vertically adjacent and both have value 1 in the binary image. 4-adjacency can create the "thin diagonal" connectivity paradox.
- **8-adjacency**: Two foreground pixels P and Q are 8-adjacent if Q belongs to $N_8(P)$ — i.e., they share an edge or corner. All 8 surrounding pixels count.
- **M-adjacency (Mixed)**: P and Q are m-adjacent if $Q \in N_4(P)$, OR $Q \in N_D(P)$ AND $N_4(P) \cap N_4(Q)$ contains no foreground pixels. M-adjacency eliminates ambiguity in diagonal paths.

4-adjacency is conservative; 8-adjacency is more liberal; M-adjacency resolves ambiguity.

TOPIC GROUP 2: Binary Shape Analysis & Connectedness

[Part A | Q6 | 5 Marks] What is a binary image?

Answer: A binary image is a digital image where each pixel takes only one of two values: **0 (background)** or **1 (foreground/object)**. It is typically obtained by thresholding a grayscale image at a value T:

> Binary(x,y) = 1 if I(x,y) ≥ T, else 0

Binary images are used extensively for:

- Shape analysis (object area, perimeter, orientation)

- Morphological operations (erosion, dilation, opening, closing)
- Connected component labeling to count and label distinct objects
- Skeleton extraction and thinning

The simplicity of two intensity levels makes binary images computationally efficient for object analysis.

[Part A | Q7 | 5 Marks] Define connected component.

Answer: A connected component (CC) is a maximal set of foreground pixels (value 1) in a binary image where every pair of pixels is connected through a continuous path of adjacent foreground pixels. Two connectivity definitions apply:

- **4-connected CC**: Each step in the path uses 4-adjacency.
- **8-connected CC**: Each step uses 8-adjacency.

Example: In a binary image, all 1-pixels forming a letter "A" form one connected component. Different letters form different CCs. Each connected component represents one distinct object or blob. The number of CCs equals the number of objects in the scene.

[Part A | Q8 | 5 Marks] What is object labeling?

Answer: Object labeling (Connected Component Labeling, CCL) assigns a unique integer label to each connected component in a binary image. The standard **Two-Pass Algorithm**:

- **Pass 1**: Scan image left-to-right, top-to-bottom. For each foreground pixel, check its already-scanned neighbours:
  - If no labelled neighbours → assign new label.
  - If one labelled neighbour → copy its label.
  - If multiple labelled neighbours → assign smallest label, record equivalences.
- **Pass 2**: Resolve all label equivalences (union-find), relabel all pixels.

Output: Each distinct object gets a unique label (1, 2, 3, …). Used as a prerequisite for shape analysis, size filtering, and feature extraction.

[Part A | Q9 | 5 Marks] What is size filtering?

Answer: Size filtering is a post-CCL operation that removes connected components based on their **pixel area** (number of pixels). After labeling: Compute the area (pixel count) of each CC.

1. Remove CCs whose area < minimum threshold (eliminates small noise blobs).
2. Optionally remove CCs whose area > maximum threshold (eliminates large irrelevant regions).

Applications:

- Removing salt-and-pepper noise in document images (small isolated 1-pixel components).
- Retaining only objects within a meaningful size range in industrial inspection.
- Pre-processing step before shape analysis to focus on objects of interest.

[Part A | Q10 | 5 Marks] Define distance transform.

Answer: The distance transform (DT) of a binary image assigns to each **foreground pixel** the minimum distance to the nearest **background (0) pixel**. Three common distance metrics:

Metric	Formula	Shape of iso-distance contours
Euclidean D_2	$\sqrt{((x_1-x_2)^2+(y_1-y_2)^2)}$	Circles
City Block D_4	$\lvert x_1-x_2\rvert+\lvert y_1-y_2\rvert$	Diamonds
Chessboard D_8	$\max(\lvert x_1-x_2\rvert, \lvert y_1-y_2\rvert)$	Squares

Application in skeleton extraction: The local maxima of the distance transform correspond to the **Medial Axis** (skeleton) of the object — points equidistant from two or more boundary points.

[Part B | B1 | 15 Marks] Explain connected component labeling algorithm with example.

Answer:

Definition: Connected Component Labeling (CCL) identifies and labels distinct objects (connected regions) in a binary image.

Two-Pass Algorithm Steps:

Pass 1 — Provisional Labeling: Scan image raster-order (left→right, top→bottom). For each foreground pixel P:

- Check already-scanned neighbours (top, left; and top-left, top-right for 8-connectivity).
- **No foreground neighbours**: assign new label, increment counter.
- **One foreground neighbour**: copy neighbour's label.
- **Multiple foreground neighbours with same label**: copy label.
- **Multiple foreground neighbours with different labels**: assign minimum label; record all label pairs as **equivalent**.

Pass 2 — Equivalence Resolution:

- Build equivalence sets (union-find data structure).
- Replace all provisional labels with the canonical label (root of equivalence set).

Worked Example (4-connectivity):

Input binary image:

```
0 0 0 0 0
0 1 1 0 0
0 1 0 0 1
0 1 1 0 1
0 0 0 0 0
```

Pass 1 result (provisional labels):

```
0 0 0 0 0
0 1 1 0 0
0 1 0 0 2
0 1 1 0 2
0 0 0 0 0
```

No equivalences → Pass 2 unchanged.

Final output: Label 1 (left object, 5 pixels), Label 2 (right object, 2 pixels).

Complexity: O(n) where n = number of pixels.

Applications: Object counting, shape analysis, size filtering, feature extraction.

[Part B | B2 | 15 Marks] Compare 4-adjacency, 8-adjacency, and m-adjacency with illustrations.

Answer:

1. 4-Adjacency:

- Pixel P at (x,y) is 4-adjacent to pixels at (x±1,y) and (x,y±1) only.
- 4-neighbours $N_4(P)$ = {(x-1,y), (x+1,y), (x,y-1), (x,y+1)}

```
  [N]
[W][P][E]
  [S]
```

- **Advantages**: Simple, avoids diagonal ambiguity, standard in most morphological ops.
- **Disadvantages**: A diagonal 1-pixel chain is NOT 4-connected even though visually connected.

2. 8-Adjacency:

- Pixel P is 8-adjacent to all 8 surrounding pixels (4 edge-sharing + 4 diagonal).
- $N_8(P) = N_4(P) \cup N_D(P)$ where N_D = {diagonals}

```
[NW][N][NE]
[W ][P][E ]
[SW][S][SE]
```

- **Advantages**: Captures diagonal connections; fewer, larger connected components.
- **Disadvantages**: Can create ambiguities — two crossing diagonal lines may appear connected; dual connectivity problem.

3. M-Adjacency (Mixed):

- P and Q are m-adjacent if:
  - $Q \in N_4(P)$, **OR**
  - $Q \in N_D(P)$ **AND** $N_4(P) \cap N_4(Q)$ contains **no foreground pixels**.
- M-adjacency uses diagonal connection only when no 4-connected path exists.
- **Advantages**: Eliminates the ambiguity of 8-adjacency; no "spurious" diagonal connections.
- **Disadvantages**: More complex to compute.

Comparison Table:

Property	4-Adjacency	8-Adjacency	M-Adjacency
Neighbours considered	4	8	4 or 4+diagonal (conditional)
# of CCs (typically)	More (splits diagonals)	Fewer (merges diagonals)	Intermediate
Diagonal connection	Never	Always	Only if necessary
Ambiguity	Low	High	None
Use case	Morphological ops	General connectivity	Unambiguous paths

Rule of thumb: When foreground uses 4-connectivity, background uses 8-connectivity, and vice versa, to maintain topological consistency (Jordan curve theorem analog).

[Part B | B3 | 15 Marks] Explain distance transform and its applications in skeleton extraction.

Answer:

Definition: The Distance Transform (DT) assigns to every foreground pixel in a binary image its minimum distance to the nearest background pixel.

Three Distance Metrics:

1. **Euclidean (D_2)**: $D(p,q) = \sqrt{((x_1-x_2)^2+(y_1-y_2)^2)}$ — circular iso-contours, most accurate.
2. **City Block (D_4)**: $D(p,q) = |x_1-x_2|+|y_1-y_2|$ — diamond-shaped iso-contours, fast to compute.
3. **Chessboard (D_8)**: $D(p,q) = \max(|x_1-x_2|,|y_1-y_2|)$ — square iso-contours.

Algorithm for D_4 DT:

- **Forward pass** (top-left to bottom-right): update each pixel as min of itself and (left+1, top+1).
- **Backward pass** (bottom-right to top-left): update using right+1 and bottom+1.
- Two passes give the complete D_4 distance transform.

Example:

Input:

```
0 0 0 0 0
0 1 1 1 0
```

```
0 1 1 1 0
0 0 1 0 0
0 0 0 0 0
```

Distance Transform (D_4):

```
0 0 0 0 0
0 1 2 1 0
0 1 2 1 0
0 0 1 0 0
0 0 0 0 0
```

Skeleton Extraction via DT (Medial Axis Transform):

1. Compute distance transform.
2. Extract **local maxima** — pixels with DT value greater than all their neighbours.
3. The set of local maxima forms the **Medial Axis** (skeleton).

For the example above, the local maxima are the pixels with value 2 — forming the centerline (skeleton) of the object.

Applications of Distance Transform:

- **Skeleton extraction** (centerline of shapes) — used in OCR, fingerprint matching.
- **Path planning** — DT gives clearance from obstacles.
- **Shape analysis** — DT values encode shape thickness.
- **Morphological erosion** — eroding by r pixels = zeroing all DT pixels < r.
- **Watershed segmentation** — DT guides flood-fill from local maxima.

[Part B | B4 | 15 Marks] Describe thinning algorithm with steps.

Answer:

Definition: Thinning is an iterative morphological operation that removes boundary pixels from foreground objects while preserving connectivity, until a 1-pixel-wide structure (skeleton) remains.

Key Principles:

- A pixel can be removed only if: (1) it is a foreground boundary pixel; (2) removing it does not disconnect its component; (3) it is not an end-point of a thin line.

Zhang-Suen Thinning Algorithm:

Operates in two sub-iterations per pass on a 3×3 neighbourhood:

```
P9 P2 P3
P8 P1 P4
P7 P6 P5
```

Conditions for deleting P_1 (value 1):

Sub-iteration 1 — delete P_1 if ALL of:

1. $2 \le B(P_1) \le 6$ — (B = number of non-zero neighbours)
2. $A(P_1) = 1$ — (A = number of 0→1 transitions in $P_2, P_3, \ldots, P_9, P_2$ sequence)
3. $P_2 \cdot P_4 \cdot P_6 = 0$
4. $P_4 \cdot P_6 \cdot P_8 = 0$

Sub-iteration 2 — delete P_1 if ALL of:

1. $2 \le B(P_1) \le 6$
2. $A(P_1) = 1$
3. $P_2 \cdot P_4 \cdot P_8 = 0$
4. $P_2 \cdot P_6 \cdot P_8 = 0$

Algorithm Steps:

1. Initialize: input binary image B.
2. Mark all pixels satisfying Sub-iteration 1 conditions as "to delete".
3. Delete all marked pixels simultaneously (parallel update).
4. Mark all pixels satisfying Sub-iteration 2 conditions as "to delete".
5. Delete all marked pixels simultaneously.
6. Repeat steps 2–5 until no pixels are deleted in a full pass.
7. Output: thinned (skeletal) image.

Example:

Input (a rectangular block):

```
1 1 1 1 1
1 1 1 1 1
1 1 1 1 1
```

After thinning:

```
0 0 0 0 0
1 1 1 1 1
0 0 0 0 0
```

(1-pixel-wide horizontal line)

Guo-Hall Algorithm: Alternative thinning using a single set of deletion conditions per pass; more efficient. Uses similar 3×3 neighbourhood analysis.

Properties of Zhang-Suen Thinning:

- Topology-preserving (connectivity maintained).
- 8-connected output.
- Parallel (all deletions in one sub-iteration are simultaneous).
- Converges in finite iterations.

Applications: OCR (character recognition), fingerprint matching, blood vessel extraction, road network analysis.

TOPIC GROUP 3: Skeletons, Thinning & Boundary

[Part A | Q11 | 5 Marks] What is skeletonization?

Answer: Skeletonization reduces a binary object to a 1-pixel-wide **skeleton** that preserves topology (connectivity and holes) and approximate shape. Three main methods:

1. **Morphological Skeletonization**: Iterative erosion — repeatedly erode the object and union the difference with the opening at each scale: $S(A) = \cup_k [(A \ominus kB) - (A \ominus kB) \circ B]$
2. **Distance Transform-Based**: Compute DT, extract local maxima → Medial Axis.
3. **Medial Axis Transform (MAT)**: Set of all skeleton branch points equidistant from two or more boundary points.

Applications: Shape representation, OCR, fingerprint recognition, vessel tree extraction.

[Part A | Q12 | 5 Marks] What is thinning?

Answer: Thinning is an iterative morphological operation that **removes boundary pixels** from foreground objects without breaking connectivity, until a 1-pixel-wide structure remains. Unlike skeletonization which finds the medial axis mathematically, thinning operates iteratively using local 3×3 neighbourhood rules.

Key algorithms: Zhang-Suen and Guo-Hall thinning algorithms.

Properties: Topology-preserving, parallel processing, produces connected 8-connected skeleton.

Applications: OCR pre-processing, fingerprint feature extraction, handwriting analysis, shape simplification.

[Part A | Q13 | 5 Marks] State one difference between thinning and skeletonization.

Answer:

- **Thinning**: An iterative pixel-deletion process using local neighbourhood rules (e.g., Zhang-Suen). Removes boundary pixels one layer at a time based on connectivity-preservation conditions. The result depends on the iteration order and deletion rules.
- **Skeletonization**: Based on the **Medial Axis Transform** — the set of local maxima of the distance transform, representing centres equidistant from two or more boundary points. It is a mathematically defined locus.

Key difference: Thinning is a topology-preserving boundary erosion process; skeletonization is the mathematically defined medial axis, computed via the distance transform. Thinning is more robust to boundary noise; skeletonization is more sensitive to boundary irregularities but is more precise geometrically.

[Part A | Q14 | 5 Marks] Define deformable shape model.

Answer: A deformable shape model is a parametric or statistical model that can represent and track shapes that **change over time or vary across instances**. It captures shape flexibility through deformation parameters.

Types:

- **Active Contour Models (Snakes)**: Curves that deform under image forces.
- **Active Shape Models (ASM)**: Use PCA to learn valid shape variations from training data.
- **Level Set Methods**: Represent shape as the zero-level set of a higher-dimensional function.
- **Statistical Shape Models (SSM/PDM)**: Model mean shape + principal modes of variation.

Applications: Medical image segmentation (organ boundaries in MRI/CT), face landmark detection, gesture recognition, motion tracking.

[Part A | Q15 | 5 Marks] What is boundary tracking?

Answer: Boundary tracking (contour tracing) is the process of extracting the ordered sequence of boundary pixels of an object in a binary image. It traverses the object perimeter systematically.

Algorithms:

1. **Moore-Neighbor Tracing** (8-connected): Starts from a boundary pixel, examines 8-neighbours clockwise, follows the boundary.
2. **Square Tracing** (4-connected): Uses only 4-neighbours, turns left on foreground, right on background.
3. **Radial Sweep**: From each boundary pixel, sweeps a radial line to find the next boundary pixel.

Output: Ordered boundary pixel sequence, often encoded as **Chain Code** (directions 0–7 for 8-connectivity).

Applications: Shape analysis, object recognition, area/perimeter computation, active contour initialization.

TOPIC GROUP 4: Active Contours & Shape Models

[Part A | Q16 | 5 Marks] What are active contours (snakes)?

Answer: Active Contours (Snakes) are deformable curves $v(s) = (x(s), y(s))$, $s \in [0,1]$ that evolve through an image under the influence of:

- **Internal forces**: Maintain smoothness and continuity of the contour.

- **External forces**: Attract the contour toward image features (edges, gradients).

The snake minimizes total energy:

> **E_snake = ∫(E_internal + E_external) ds**

where E_internal = αE_elastic + βE_smooth (first and second derivative terms).

The contour evolves iteratively until energy is minimized — converging on object boundaries. Introduced by Kass, Witkin, and Terzopoulos (1988).

[Part A | Q17 | 5 Marks] Mention two applications of active contours.

Answer:

1. **Medical Image Segmentation**: Active contours detect organ boundaries (heart, liver, tumour) in MRI, CT, and X-ray images. An initial contour placed near the organ evolves to fit the boundary automatically. Used in Active Shape Models (ASM) for detecting facial landmarks (eyes, nose, lips).
2. **Object Tracking in Video**: A contour initialized around an object in the first frame is updated per frame using optical flow and image gradients to track moving objects (pedestrian tracking, vehicle tracking). The snake adapts to shape changes and deformations across frames.

[Part A | Q18 | 5 Marks] Define shape recognition.

Answer: Shape recognition is the process of **identifying and classifying shapes** in an image based on learned patterns or geometric properties.

Approaches (from slides):

1. **Template Matching**: Compare input with stored templates — works for predefined shapes.
2. **Feature-Based Recognition**: Use contour features (SIFT/SURF descriptors).
3. **Machine Learning / Deep Learning**: CNNs classify shapes from training data.
4. **Shape Context Matching**: Compare spatial distribution of points around keypoints.
5. **Fourier Descriptors**: Transform boundary into frequency domain; rotation/scale-invariant coefficients.

Applications: Handwritten character recognition (OCR), medical shape analysis, gesture recognition, object detection.

[Part A | Q19 | 5 Marks] What is a statistical shape model?

Answer: A Statistical Shape Model (SSM) learns the distribution of shape variations from a training dataset using statistical methods (typically PCA). Any shape is expressed as:

> **$x = \bar{x} + P \cdot b$**

where $\bar{x}$ = mean shape, P = matrix of principal modes of variation, b = shape parameter vector.

Types:

- **Active Shape Models (ASM)**: Extension of active contours using PCA-learned shape constraints.
- **Point Distribution Models (PDM)**: Landmarks placed on shapes, statistics computed.

Use: Constrains deformation to plausible shapes during segmentation, preventing unrealistic contours.

Applications: Organ shape analysis in MRI, face detection and alignment, cardiac segmentation.

[Part A | Q20 | 5 Marks] What is Hausdorff distance?

Answer: Hausdorff distance measures the **maximum dissimilarity** between two sets of points A and B. It is defined as:

> **H(A, B) = max(h(A,B), h(B,A))**

where the directed Hausdorff distance:

> h(A, B) = max_{a∈A} min_{b∈B} d(a, b)

h(A,B) finds the point in A that is farthest from any point in B.

Interpretation: The Hausdorff distance is the smallest ε such that every point in A is within ε of some point in B, and vice versa.

Properties: Symmetric, satisfies triangle inequality, sensitive to outliers.

Applications: Shape matching and retrieval, comparing binary image objects, object recognition, evaluating segmentation quality against ground truth.

[Part B | B5 | 15 Marks] Explain active contour model with energy formulation.

Answer:

One-line idea: A snake is a rubber band placed near an object — it contracts and snaps onto the object's edge by minimizing energy.

Definition

An active contour (snake) is a deformable curve **v(s) = (x(s), y(s))** that moves through an image until it settles on an object boundary. Introduced by Kass, Witkin & Terzopoulos (1988).

Energy — The Snake Minimizes 3 Things

> **E_total = E_internal + E_external**

Think of it as: the snake wants to be *smooth* (internal) AND *near edges* (external). It keeps moving until both are satisfied.

Internal Energy — "How well-behaved is the curve?"

> **$E_internal = \alpha|v'|^2 + \beta|v''|^2$**

Term	What it measures	Effect
$\alpha\|v'\|^2$ (1st derivative)	Stretching / spacing	High α → points stay evenly spaced (elastic)
$\beta\|v''\|^2$ (2nd derivative)	Bending / curvature	High β → curve stays smooth (stiff)

- **Memory trick:** α = *elasticity*, β = *bending*. First derivative → stretch. Second derivative → bend.

External Energy — "How close is the curve to edges?"

> **$E_external = -|\nabla(G_\sigma * I)|^2$**

- Smooth image with Gaussian G_σ → compute gradient (edge strength) → negate it.
- **High gradient = strong edge = low energy → snake gets pulled there.**
- **Memory trick:** Edges are valleys. The snake rolls downhill into them.

How It Works — 4 Steps

1. **Place** the initial curve near the object.
2. **Compute** forces: internal (smoothness) + external (edge pull).
3. **Move** the curve: `v_new = v_old - γ·∇E_total`
4. **Repeat** until the curve stops moving → it's on the boundary.

Parameter Cheat Sheet

Parameter	What it does
High α	Stiff spring — resists stretching
High β	Smooth rod — resists bending
High σ	Wider edge pull range, less noise sensitive
Low α, low β	Floppy snake — follows complex shapes

Limitation + Fix

- **Problem:** Snake gets stuck if placed too far from the edge (local minimum).
- **Fix:** GVF (Gradient Vector Flow) — spreads edge forces further out so the snake can start from farther away.

[Part B | B6 | 15 Marks] Discuss internal and external energies in snake model.

Answer:

Overview: The Snake model deforms a curve iteratively by minimizing the sum of internal and external energies: **E_snake = E_internal + E_external**.

INTERNAL ENERGY:

Internal energy keeps the contour **smooth and continuous**, preventing physically unrealistic deformations.

Formula:

> $E_{internal} = \alpha(s)|v'(s)|^2 + \beta(s)|v''(s)|^2$

Components:

A. Elasticity Energy (First Order Term): $\alpha|v'(s)|^2$

- $v'(s)$ = first derivative of the contour (spacing between consecutive points).
- Penalizes unequal spacing → acts like a spring connecting adjacent points.
- High α: contour resists stretching/compression → point spacing stays uniform.
- Low α: contour can bunch up or stretch.

B. Bending/Smoothness Energy (Second Order Term): $\beta|v''(s)|^2$

- $v''(s)$ = second derivative = curvature of the contour.
- Penalizes sharp bends → promotes smooth, curved contours.
- High β: very smooth contour (like stiff metal rod).
- Low β: allows sharp corners (like rubber band).
- $\beta = 0$ at a point: allows a corner at that point (used for sharp feature detection).

EXTERNAL ENERGY:

External energy **attracts the contour toward salient image features** — primarily edges.

Formula:

> $E_{external} = E_{image}(v(s)) = -|\nabla(G_\sigma * I)|^2$

where $G_\sigma * I$ is the Gaussian-smoothed image, and ∇ denotes the gradient operator.

Components:

A. Image Gradient Energy:

> $E_{image} = -|\nabla I|^2$

- Negative sign: energy is minimized where gradients are strong.
- Edge pixels have high $|\nabla I|$ → low E_{image} → attraction.
- Flat regions have low $|\nabla I|$ → high E_{image} → repulsion.

B. Gaussian Smoothing: $G_\sigma * I$

- Pre-smooth image to extend the pull range of edges (larger σ = wider capture range).
- Without smoothing, the snake only "feels" edges in its immediate vicinity.

C. Line/Termination Energy (optional):

- $E_{line} = I(x,y)$: attracts to bright/dark lines.

- E_term: attracts to line endings and corners.

Interaction of Internal and External Energies:

Scenario	Internal Energy	External Energy	Outcome
High α, high β	Strong	Weak	Contour barely deforms — stays near initial position
Low α, low β	Weak	Strong	Contour wraps tightly around edges
High β	Strong smoothing	Moderate	Smooth contour, misses fine details
β = 0 at a point	Allows corner	Strong	Detects sharp corners

Energy Minimization: The contour v evolves by gradient descent on total energy until convergence (no further significant movement).

[Part B | B7 | 15 Marks] Explain deformable shape analysis for medical image segmentation.

Answer:

Definition: Deformable Shape Analysis uses models that can flex and adapt to segment variable anatomical structures in medical images. The model is initialized near a target organ and deforms under image forces + shape constraints.

Why Needed in Medical Imaging:

- Organs vary in shape, size, and position across patients.
- Noise, low contrast, and similar intensities of adjacent tissues make thresholding insufficient.
- Deformable models provide anatomically constrained segmentation.

Types of Deformable Models Used:

1. Active Contours (Snakes) — 2D:

- Initialize contour around organ (e.g., manually or using atlas).
- Minimize E_snake = E_internal + E_external.
- E_internal enforces smoothness; E_external attracts to organ boundaries (MRI gradient).
- **Application**: Cardiac ventricle segmentation, tumour boundary detection.

2. Active Shape Models (ASM) — Statistical:

- **Training phase**: Mark N landmarks on each training image. Compute mean shape $\bar{x}$ and PCA modes P.
- **Fitting phase**: Initialize mean shape → iteratively move landmarks toward nearest strong edges → project back to allowed shape space using: $b = P^T(x - \bar{x})$, clamp $|b_i| \leq 3\sqrt{\lambda_i}$.
- **Application**: Cardiac shape fitting in echocardiograms, vertebra detection in X-rays.

3. Level Set Methods:

- Represent boundary as the zero level set of a 3D surface $\phi(x,y,t)$: contour = $\{(x,y): \phi(x,y,t) = 0\}$.
- Evolves according to: $\partial\phi/\partial t + F|\nabla\phi| = 0$ where F = speed function based on image features.
- **Advantage**: Handles topology changes automatically (splitting, merging).
- **Application**: 3D organ segmentation in CT scans.

Workflow for Medical Image Segmentation:

1. **Pre-processing**: Gaussian denoising, contrast enhancement.
2. **Initialization**: Place initial contour/model near target organ (manual or atlas-based).
3. **Energy/Force Computation**: Compute gradient map, internal forces.
4. **Iteration**: Deform contour minimizing energy until convergence.
5. **Post-processing**: Smooth boundary, fill holes, compute volume/area.

Applications:

- Liver segmentation in CT for surgical planning.
- Brain tumour delineation in MRI.
- Prostate segmentation in ultrasound.
- Cardiac wall motion analysis.

Advantage over fixed models: Adapts to patient-specific shape variations while remaining constrained to anatomically plausible shapes.

TOPIC GROUP 5: Feature Detection & Matching

[Part A | Q21 | 5 Marks] Define feature detector.

Answer: A feature detector is an algorithm that identifies distinct, **stable, and repeatable locations (keypoints)** in an image — locations likely to match well across different images.

Characteristics of a Good Feature (from slides):

- **Repeatable**: Same feature detectable in different images of same scene.
- **Distinctive**: Unique, distinguishable from surrounding areas.
- **Invariant**: Stable under changes in scale, rotation, illumination, perspective.
- **Efficient**: Fast to compute for real-time applications.

Types of Features:

- **Corner features**: Sharp intensity change in multiple directions (Harris, FAST).
- **Blob features**: Region with different intensity from surroundings (SIFT DoG).
- **Edge features**: High gradient magnitude locations.

Popular detectors: Harris Corner, SIFT, SURF, ORB, FAST.

[Part A | Q22 | 5 Marks] What is feature descriptor?

Answer: A feature descriptor encodes the appearance of the local patch around a detected keypoint into a **compact numerical vector** that can be compared across images.

Steps in descriptor extraction:

1. Extract a small patch around each keypoint.
2. Compute gradient magnitudes and orientations.
3. Normalize the descriptor for scale and rotation invariance.
4. Convert to a numerical vector for comparison.

Example — SIFT Descriptor:

- Extract 16×16 patch around keypoint.
- Divide into 4×4 sub-blocks.
- Each sub-block generates an 8-bin histogram of gradient orientations.
- Final descriptor: **128-dimensional vector** (4×4×8 = 128).
- SURF uses 64-dim (Haar wavelets); ORB uses 32-byte binary descriptor.

[Part A | Q23 | 5 Marks] What is feature matching?

Answer: Feature matching compares descriptors from two or more images to find **corresponding keypoints** (same physical point seen in different images).

Steps:

1. Compute distance between descriptor pairs (Euclidean for SIFT/SURF, Hamming for ORB).
2. Find the best (nearest-neighbour) match for each descriptor.
3. Filter false matches using **Lowe's Ratio Test**: accept match only if $d_1/d_2 < 0.75$ (ratio of nearest to second-nearest distance).

Methods:

- **BFMatcher** (Brute-Force): Compares every descriptor in image A with every descriptor in image B — accurate but slow.
- **FLANN** (Fast Library for Approximate Nearest Neighbors): Uses k-d trees for fast matching.

Applications: Image stitching, 3D reconstruction, object recognition, SLAM.

[Part A | Q24 | 5 Marks] Name two popular feature detectors.

Answer:

1. Harris Corner Detector:

- Detects corners based on intensity variation in all directions.
- Computes second moment matrix $M = [[I_x^2, I_xI_y],[I_xI_y, I_y^2]]$.
- Response function: $\mathbf{R = \det(M) - k\cdot(\mathrm{trace}(M))^2}$, $k \in [0.04, 0.06]$.
- If $R > T$: corner; $R \approx 0$: flat; $R < 0$: edge.
- **Not scale-invariant** — fails under scaling.

2. SIFT (Scale-Invariant Feature Transform):

- Detects keypoints invariant to **scale and rotation** using Difference-of-Gaussian (DoG) pyramid.
- Generates a **128-dimensional descriptor** from gradient orientations.
- Robust to illumination changes and partial affine transformations.
- Used for object recognition, image stitching, 3D reconstruction.

[Part A | Q25 | 5 Marks] What is feature tracking?

Answer: Feature tracking ensures detected keypoints **persist and are followed across multiple frames** in a video sequence without re-detecting in every frame.

Working Principle (from slides):

1. Detect features in the first frame (using Harris/SIFT).
2. Track movement across frames using **optical flow** (Lucas-Kanade method).
3. Update keypoint positions in real-time within a small search window.

Algorithms:

- **Lucas-Kanade Optical Flow**: Tracks features by comparing small patches in consecutive frames. Assumes constant intensity (brightness constancy).
- **KLT (Kanade-Lucas-Tomasi) Tracker**: Uses pyramidal optical flow for robust real-time tracking.
- **SIFT-based Tracking**: Matches SIFT features in consecutive frames.

Applications: Video stabilization, motion detection, SLAM (Simultaneous Localization and Mapping), augmented reality.

[Part B | B8 | 15 Marks] Explain feature detection and feature description pipeline.

Answer:

Overview: The keypoint detection and matching pipeline transforms raw images into matched feature pairs for tasks like image stitching, object recognition, and 3D reconstruction.

4-Stage Pipeline (from slides):

STAGE 1: Feature Detection

- Search each image for stable, repeatable locations.
- **Goal**: Find pixels that are distinctive and likely to match across images.
- **Basic Algorithm:**
    1. Compute horizontal (Ix) and vertical (Iy) gradients by convolving with derivative-of-Gaussian.
    2. Compute outer product images (Ix^2, IxIy, Iy^2) — second moment matrix A.
    3. Convolve each with a Gaussian for spatial averaging.
    4. Compute scalar interest measure (Harris $R = \det(A) - k \cdot \mathrm{trace}(A)^2$).
    5. Find local maxima above threshold → detected keypoints.

Output: Set of keypoint locations $\{(x_i, y_i)\}$.

STAGE 2: Feature Description

- For each keypoint, compute a compact, invariant descriptor.
- **SIFT Descriptor Construction:**
    1. Find dominant orientation (from gradient histogram in 36-bin circle around keypoint).
    2. Rotate patch to canonical orientation (rotation invariance).
    3. Extract 16×16 patch at keypoint scale.
    4. Divide into 4×4 sub-blocks.
    5. Compute 8-bin gradient orientation histogram per sub-block.
    6. Concatenate → 128-dimensional vector.
    7. Normalize → robust to illumination.

Output: Descriptor vectors $\{d_i\}$ for each keypoint.

STAGE 3: Feature Matching

- Find correspondences between keypoint descriptors of two images.
- **Distance Metrics**: Euclidean distance for SIFT/SURF; Hamming distance for ORB.
- **Methods**:
    - BFMatcher: exhaustive comparison — O(nm) for n and m keypoints.
    - FLANN: approximate nearest-neighbour using k-d trees — O(n log m).
- **Lowe's Ratio Test**: Match (i,j) accepted if $d(d_i,d_{j_1})/d(d_i,d_{j_2}) < 0.75$.

Output: Set of matched pairs {(keypoint_A, keypoint_B)}.

STAGE 4: Feature Tracking (Video)

- Instead of re-detecting in every frame, track keypoints across frames.
- Search only a small neighbourhood around each detected feature (Lucas-Kanade window).
- Update positions each frame using optical flow constraint:

> $I_x u + I_y v + I_t = 0$

Output: Keypoint trajectories across frames.

Summary Diagram:

```
Image A, B
   ↓
[Detection] → Keypoints A, Keypoints B
   ↓
[Description] → Descriptors A, Descriptors B
   ↓
[Matching] → Matched pairs
   ↓
[Geometric Verification] → Inlier matches (RANSAC)
```

[Part B | B9 | 15 Marks] Compare feature detectors: Harris Corner Detector, SIFT, SURF.

Answer:

---

1. Harris Corner Detector:

Concept: Detects corners by analyzing intensity variation in all directions using the second moment matrix.

Algorithm:

1. Compute Ix, Iy using Sobel filters.
2. Build structure tensor: $M = [[\Sigma Ix^2, \Sigma IxIy], [\Sigma IxIy, \Sigma Iy^2]]$ (summed over window W).
3. Compute response: $\mathbf{R = det(M) - k \cdot (trace(M))^2}$
   - $det(M) = Ix^2 \cdot Iy^2 - (IxIy)^2$
   - $trace(M) = Ix^2 + Iy^2$
   - $k \in [0.04, 0.06]$
4. Threshold R: R > T → corner; R ≈ 0 → flat; R < 0 → edge.
5. Non-maximum suppression to select local maxima.

Interpretation of eigenvalues λ_1, λ_2:

- Both small → flat region
- $\lambda_1 >> \lambda_2$ → edge
- Both large → corner (R > 0)

Properties: Rotation invariant (R is invariant to rotation), NOT scale invariant, fast.

---

2. SIFT (Scale-Invariant Feature Transform):

Concept: Detects keypoints invariant to scale AND rotation using a scale-space pyramid.

Algorithm:

1. **Scale-Space Construction**: Build Gaussian pyramid (σ, 2σ, 4σ...). Compute Difference of Gaussian ($DoG = G(\sigma_{k+1})*I - G(\sigma_k)*I$).
2. **Keypoint Detection**: Find local extrema in DoG across scale and space (26 neighbours: 8 in-plane + 9 above + 9 below).
3. **Keypoint Localization**: Refine using Taylor expansion; discard low-contrast points and edge responses.
4. **Orientation Assignment**: Compute gradient histogram in 36 bins (10° each) around keypoint; dominant orientation → rotation invariance.
5. **Descriptor Generation**: 16×16 patch → 4×4 sub-blocks → 8-bin histograms → **128-dim vector**; normalize.
6. **Matching**: Euclidean distance + Lowe's ratio test.

Properties: Scale invariant, rotation invariant, partial illumination invariant. Slow (128-dim descriptor). Patented (now open).

---

3. SURF (Speeded-Up Robust Features):

Concept: Faster approximation of SIFT using integral images and box filters.

Algorithm:

1. **Scale-Space**: Uses **Hessian matrix** with **box filters** (approximation of Gaussian derivatives) — faster than DoG.
2. **Keypoint Detection**: Local maxima of Hessian determinant across scales.
3. **Orientation Assignment**: Use **Haar wavelets** (horizontal + vertical) in circular neighbourhood → dominant orientation.
4. **Descriptor**: 4×4 sub-regions, each encoded with Haar wavelet responses (dx, dy, |dx|, |dy|) → **64-dimensional vector**.
5. **Matching**: Euclidean distance.

Properties: ~3× faster than SIFT. Scale and rotation invariant. 64-dim descriptor (vs 128 for SIFT). Uses integral images for O(1) box filter computation.

Comparison Table:

Property	Harris	SIFT	SURF
Scale invariance	No	Yes (DoG pyramid)	Yes (Hessian)
Rotation invariance	Yes	Yes	Yes
Illumination invariance	Partial	Yes	Yes
Descriptor dimension	None (detector only)	128-dim	64-dim
Gradient computation	Sobel	DoG	Haar wavelet
Speed	Fast	Slow	Medium (3× faster than SIFT)
Feature type	Corners only	Corners + blobs	Corners + blobs
Noise robustness	Low	High	High
Patent	Open	Patented (now free)	Patented

[Part B | B10 | 15 Marks] Explain feature matching using Euclidean distance and ratio test.

Answer:

Definition: Feature matching finds correspondences between keypoints in two images by comparing their descriptors using a similarity (or distance) measure.

STEP 1: Descriptor Distance Computation

For **SIFT and SURF** (real-valued descriptors):

> **Euclidean Distance**: $d(dA, dB) = \sqrt{(\Sigma_i (dA_i - dB_i)^2)}$

For a 128-dim SIFT descriptor:

> $d = \sqrt{(\Sigma_{i=1}^{128} (dA_i - dB_i)^2)}$

For **ORB** (binary descriptors):

> **Hamming Distance**: count of bit positions where dA and dB differ (XOR + popcount).

STEP 2: Nearest-Neighbour Matching

For each descriptor dA_i in image A:

1. Find the **closest descriptor** dB_1 in image B: $d_1 = \min_j\ d(dA_i, dB_j)$.
2. Find the **second-closest descriptor** dB_2 in image B: $d_2 = \text{second_min_j}\ d(dA_i, dB_j)$.

STEP 3: Lowe's Ratio Test (False Match Rejection)

Proposed by David Lowe (SIFT paper, 2004):

> **Accept match if: $d_1 / d_2 < 0.75$** (threshold typically 0.7–0.8)

Intuition:

- If $d_1 \approx d_2$: the descriptor matches two different keypoints almost equally well → **ambiguous match → reject**.
- If $d_1 << d_2$: the descriptor strongly matches one keypoint → **reliable match → accept**.

Why this works: True matches have a distinctive nearest neighbour; false matches have multiple similar-distance candidates.

STEP 4: Geometric Verification

Even after ratio test, some false matches remain. Use **RANSAC** to fit a geometric model (homography, fundamental matrix) and keep only geometrically consistent inlier matches.

Matching Methods:

1. Brute-Force Matcher (BFMatcher):

- Compares every descriptor in image A with every descriptor in image B.
- Complexity: $O(n \cdot m)$ where n, m = number of keypoints.
- Most accurate; slowest.

2. FLANN (Fast Library for Approximate Nearest Neighbors):

- Builds a k-d tree on descriptors of image B.
- Queries k-d tree for each descriptor in A.
- Approximate search: much faster for large descriptor sets.
- Complexity: $O(n \log m)$.

Summary of Pipeline:

```
Descriptors A → [BFMatcher / FLANN] → Raw matches
Raw matches → [Ratio Test d₁/d₂ < 0.75] → Filtered matches
Filtered matches → [RANSAC geometric check] → Final inlier matches
```

TOPIC GROUP 6: Edge Detection & Linking

[Part A | Q26 | 5 Marks] Define edge.

Answer: An edge in a digital image is a **curve (set of pixels)** along which there is a significant, local change in image intensity (brightness or colour). Edges correspond to object boundaries, surface markings, illumination discontinuities, or depth changes.

Mathematical definition: An edge point exists where the **gradient magnitude** $|\nabla I|$ is locally maximum in the gradient direction:

> $|\nabla I| = \sqrt{(I_x^2 + I_y^2)} = \sqrt{((\partial I/\partial x)^2 + (\partial I/\partial y)^2)}$

Types:

- **Step edge**: Sharp intensity transition (ideal edge).
- **Ramp edge**: Gradual transition over several pixels.
- **Ridge edge**: Narrow bright/dark stripe.
- **Roof edge**: Blunt-topped ridge.

Importance: Edges are the most fundamental low-level feature for object detection, segmentation, and shape analysis.

[Part A | Q27 | 5 Marks] What is edge linking?

Answer: Edge linking is the process of **connecting detected edge pixels** into coherent, continuous curves (contours) after edge detection produces a binary edge map with gaps and isolated pixels.

Methods:

1. **Local processing**: For each edge pixel, check if a neighbour has similar gradient magnitude and direction within thresholds. If yes, they belong to the same edge.
2. **Hough Transform**: Global method — find line/curve parameters that explain the most edge pixels simultaneously, then draw the detected line.
3. **Graph-based linking**: Model edge pixels as graph nodes; connect nodes that satisfy local criteria; extract paths.
4. **Hysteresis thresholding** (Canny): Strong edges anchor chains; weak edge pixels are linked if connected to strong edges.

Challenges: Noise creates spurious edges; occlusion creates gaps; junctions create branching decisions.

[Part A | Q28 | 5 Marks] What is successive approximation?

Answer: Successive approximation is an iterative refinement technique for **approximating curves or boundaries** with increasing precision at each step.

In the context of boundary approximation: The **Douglas-Peucker (Split-and-Merge) algorithm** is a classic successive approximation method:

1. Start with a straight line connecting two endpoints of a boundary segment.
2. Find the boundary point farthest from the line.
3. If distance > threshold: split at that point, creating two sub-segments.
4. Recurse on each sub-segment.
5. Stop when all points are within threshold of their segment.

In edge detection context: Refers to progressively computing edge responses at finer scales — start with a coarse Gaussian-smoothed image, successively increase precision.

Application: Polygon approximation of shapes, compression of boundary representations, multi-scale edge analysis.

[Part A | Q29 | 5 Marks] What is Canny edge detection?

Answer: Canny edge detection is the **optimal multi-step edge detection algorithm** (John Canny, 1986), designed to satisfy: (1) good detection, (2) good localization, (3) single response per edge.

6 Steps:

1. **Gaussian Smoothing**: $I_{smooth} = G_\sigma * I$ — reduce noise (σ controls smoothing level).
2. **Gradient Computation**: Compute $I_x = \partial I_{smooth}/\partial x$, $I_y = \partial I_{smooth}/\partial y$ using Sobel.
3. **Gradient Magnitude**: $|\nabla I| = \sqrt{(I_x^2 + I_y^2)}$.
4. **Gradient Direction**: $\theta = \arctan(I_y/I_x)$ — rounded to 0°, 45°, 90°, 135°.
5. **Non-Maximum Suppression**: Keep pixel only if it is a local maximum in gradient direction (thin edges to 1-pixel width).
6. **Double Thresholding + Hysteresis**:
   - Strong edges: $|\nabla I| > T_{high}$ → definitely an edge.
   - Weak edges: $T_{low} < |\nabla I| < T_{high}$ → edge only if connected to a strong edge.
   - $|\nabla I| < T_{low}$ → not an edge.

[Part A | Q30 | 5 Marks] What is gradient magnitude?

Answer: Gradient magnitude measures the **strength of intensity change** at each pixel location, indicating how rapidly pixel values are changing spatially.

Computation: First compute partial derivatives using Sobel operators:

- $I_x = \partial I/\partial x$ (horizontal gradient) — using Sobel $G_x = [[-1,0,1],[-2,0,2],[-1,0,1]]$
- $I_y = \partial I/\partial y$ (vertical gradient) — using Sobel $G_y = [[-1,-2,-1],[0,0,0],[1,2,1]]$

Gradient Magnitude:

> **$|\nabla I| = \sqrt{(I_x^2 + I_y^2)}$**

Approximated for efficiency as: $|\nabla I| \approx |I_x| + |I_y|$

Gradient Direction:

> $\theta = \arctan(I_y / I_x)$

Interpretation:

- High $|\nabla I|$ → strong intensity transition → likely edge.
- Low $|\nabla I|$ → smooth/flat region → no edge.

The gradient magnitude image is thresholded (or non-maximum suppressed) to produce the edge map.

[Part B | B11 | 15 Marks] Explain Canny edge detection with mathematical formulation.

Answer:

Motivation: Canny (1986) defined three optimal edge detection criteria:

1. **Good detection**: Minimize missed edges and false detections.
2. **Good localization**: Detected edge should be as close as possible to the true edge.
3. **Single response**: One detected edge point per actual edge point.

Step 1: Gaussian Smoothing

> $I_{smooth}(x,y) = G_\sigma(x,y) * I(x,y)$

where $G_\sigma(x,y) = (1/2\pi\sigma^2)\cdot\exp(-(x^2+y^2)/2\sigma^2)$

- Removes high-frequency noise.
- σ controls the smoothing level: large σ → more smoothing → detects coarser edges.

Step 2: Gradient Computation Apply Sobel operators:

> $I_x = G_\sigma * I$ convolved with $G_x = [[-1,0,1],[-2,0,2],[-1,0,1]]$ $I_y = G_\sigma * I$ convolved with $G_y = [[-1,-2,-1],[0,0,0],[1,2,1]]$

Step 3: Gradient Magnitude and Direction

> **Magnitude**: $M(x,y) = \sqrt{(I_x^2 + I_y^2)}$ **Direction**: $\theta(x,y) = \arctan(I_y/I_x)$

Direction is rounded to 4 discrete angles: **0°, 45°, 90°, 135°** (perpendicular to edge).

Step 4: Non-Maximum Suppression (NMS) For each pixel (x,y):

- Compare $M(x,y)$ with its two neighbours in gradient direction θ.
- If $M(x,y)$ is not the local maximum → suppress (set to 0).
- This **thins** edges from multi-pixel ridges to 1-pixel-wide edges.

Example: If θ = 0° (horizontal gradient), compare with left and right neighbours M(x-1,y) and M(x+1,y).

Step 5: Double Thresholding Two thresholds: T_high and T_low (typically T_high = 2×T_low):

- M(x,y) > T_high → **strong edge pixel** (definite edge)
- T_low < M(x,y) ≤ T_high → **weak edge pixel** (potential edge)
- M(x,y) ≤ T_low → **non-edge** → suppress

Step 6: Edge Tracking by Hysteresis

- Start from strong edge pixels.
- Include weak edge pixels **connected (8-connected)** to strong edge pixels.
- Discard isolated weak edge pixels (noise).

This links edges across small gaps and eliminates noise-induced isolated pixels.

Complete Mathematical Summary:

```
I → [G_σ ∗] → I_smooth → [Sobel] → (Ix, Iy)
→ [sqrt(Ix²+Iy²)] → M → [NMS] → M_thin
→ [T_high, T_low] → (strong, weak) → [Hysteresis] → Edge map
```

Advantages over Sobel/Prewitt:

- Better localization (NMS).
- Fewer false positives (hysteresis).
- User-controlled quality via σ, T_high, T_low.

[Part B | B12 | 15 Marks] Discuss edge linking techniques and challenges.

Answer:

Definition: Edge linking joins fragmented edge pixels (from edge detection) into continuous, coherent curves representing object boundaries.

TECHNIQUE 1: Local Processing (Gradient-Based Linking)

For each detected edge pixel P at (x,y):

- Examine 8-neighbours.
- Link P to neighbour Q if:
    1. |M(x,y) − M(x_Q,y_Q)| ≤ threshold T_m (similar magnitude).
    2. |θ(x,y) − θ(x_Q,y_Q)| ≤ threshold Tθ (similar direction).
- Trace the edge chain until no more valid neighbours are found.

Advantage: Simple, fast, local. **Limitation**: Cannot bridge large gaps.

TECHNIQUE 2: Hysteresis Thresholding (Canny's Method)

- Mark pixels > T_high as strong; T_low < p < T_high as weak.
- Begin edge chains at strong pixels.
- Recursively include connected weak pixels.
- **Effectively bridges small gaps** where gradient briefly drops below T_high.

Advantage: Connects naturally fragmented edges. **Limitation**: Cannot bridge gaps larger than the weak-pixel region.

TECHNIQUE 3: Hough Transform (Global Method)

- All edge pixels vote for line/curve parameters in accumulator space.
- Peaks in accumulator identify parameters of the dominant line/curve.
- All pixels voting for the peak are linked as belonging to that line.

Advantage: Handles large gaps, noisy edges. **Limitation**: Limited to parametric shapes; slow for complex shapes.

TECHNIQUE 4: Graph-Based Linking

- Model edge pixels as nodes in a graph.
- Connect nodes that satisfy proximity and orientation criteria.
- Find long paths in the graph (dynamic programming or minimum spanning tree).

Advantage: Handles arbitrary curved edges. **Limitation**: Computationally expensive.

TECHNIQUE 5: Snake / Active Contour Linking

- Initialize a snake near fragmented edges.
- Snake interpolates across gaps using internal (smoothness) energy.
- Connects discontinuous edges by evolving through image gradients.

CHALLENGES IN EDGE LINKING:

Challenge	Description	Solution
Gaps in edges	Low contrast or noise causes missing edge pixels	Hysteresis thresholding, Hough Transform
Junction handling	Multiple edges meet — which to link?	Graph-based methods with junction detection
False edges	Texture, noise produce spurious edges	Higher thresholds, smoothing
Closed contour detection	Need to detect when edge chain closes on itself	Active contours, graph cycle detection

Challenge	Description	Solution
Multiple edges	Parallel edges (e.g., road lanes)	Separate by direction/distance thresholds
Scale sensitivity	Fine details vs large structures	Multi-scale linking

---

TOPIC GROUP 7: Hough Transform, RANSAC & GHT

[Part A | Q31 | 5 Marks] Define Hough Transform (HT).

Answer: The Hough Transform (HT) is a **feature extraction technique** that detects geometric shapes (lines, circles, ellipses, arbitrary shapes) in binary edge images by transforming image space to **parameter space**.

Core Idea: Each edge pixel votes for all parameter values consistent with it lying on the target shape. The accumulator cell with the most votes corresponds to the most prominent shape.

Properties:

- Robust to noise, partial occlusion, and missing edges.
- Transforms the shape-detection problem into a peak-finding problem.
- Works on the output of an edge detector (e.g., Canny).

Applications: Lane detection, circle/ellipse detection, architectural line detection, iris localization.

[Part A | Q32 | 5 Marks] Write the polar form of a line used in HT.

Answer: The **polar (foot-of-normal) form** of a line is:

> **$\rho = x \cdot \cos(\theta) + y \cdot \sin(\theta)$**

where:

- **ρ** = perpendicular distance from the origin to the line ($-d_{max} \le \rho \le d_{max}$)
- **θ** = angle of the normal to the line ($0° \le \theta < 180°$)

Why use polar form instead of $y = mx + b$:

- Slope-intercept form fails for vertical lines ($m \to \infty$).
- Polar form is bounded — accumulator array is always finite.
- Every line can be uniquely represented.

Mapping: Each point (x_0, y_0) in image space maps to a **sinusoidal curve** in (ρ, θ) parameter space: $\rho = x_0 \cdot \cos(\theta) + y_0 \cdot \sin(\theta)$. Points on the same line map to sinusoids that intersect at a common (ρ,θ) point.

[Part A | Q33 | 5 Marks] What is accumulator array?

Answer: The accumulator array (Hough space) is a **2D grid indexed by quantized parameter values** that records votes from edge pixels for candidate shape parameters.

For line detection (ρ,θ space):

- Dimensions: $\rho \in [-d, d]$ quantized in $\Delta\rho$ steps; $\theta \in [0°, 180°)$ quantized in $\Delta\theta$ steps.
- For each edge pixel (x,y): increment $A[\rho,\theta]$ for all θ, where $\rho = x\cdot\cos(\theta) + y\cdot\sin(\theta)$.

Peak detection: After all pixels vote, find local maxima in $A[\rho,\theta]$ that exceed a threshold. Each peak represents a detected line.

For circle detection: 3D accumulator A[a,b,r] for circle centre (a,b) and radius r.

Trade-off: Finer quantization = more accurate but larger memory and slower. Coarser = faster but less accurate.

[Part A | Q34 | 5 Marks] What is foot-of-normal method?

Answer: The foot-of-normal method is the **polar parameterization of lines** in the Hough Transform using perpendicular distance (ρ) and normal angle (θ):

> **$\rho = x\cdot\cos(\theta) + y\cdot\sin(\theta)$**

The "foot" refers to the point on the line **closest to the origin** (the foot of the perpendicular from the origin to the line).

Geometric interpretation:

- Draw a perpendicular from the origin to the line.
- ρ = length of this perpendicular (signed).
- θ = angle this perpendicular makes with the x-axis.

Advantage over (m,b) parameterization:

- Handles vertical lines (θ = 90°).
- Bounded parameter space ($\rho \in [-\sqrt{(W^2+H^2)}, \sqrt{(W^2+H^2)}]$, $\theta \in [0°,180°)$).
- Natural representation for line localization — gives the closest point of the line to origin.

Application: Standard form for line detection in Hough Transform.

[Part A | Q35 | 5 Marks] Define RANSAC.

Answer: RANSAC (Random Sample Consensus) is an iterative, robust model-fitting algorithm that estimates model parameters from data **contaminated with a large proportion of outliers**.

Steps:

1. Randomly select a minimal subset of points (e.g., 2 points for a line).
2. Fit a model to the selected subset.
3. Count **inliers**: points within distance threshold ε of the model.
4. If inlier count > best-so-far, save this model.

5. Repeat N iterations.
6. Refit final model using **all inliers** of the best model.

Number of iterations:

> $N = \log(1 - P) / \log(1 - w^s)$

where P = desired confidence (0.99), w = inlier ratio, s = sample size.

Applications: Line detection, homography estimation, pose estimation, feature matching outlier rejection.

[Part A | Q36 | 5 Marks] How is circle represented in parametric form?

Answer: A circle is represented in **standard parametric form** as:

> $\mathbf{(x - a)^2 + (y - b)^2 = r^2}$

where: (a, b) = centre coordinates, r = radius.

For Circular Hough Transform (CHT): The parameter space is **3D**: accumulator A[a, b, r].

For each edge pixel (x, y) and a candidate radius r, the possible centres are:

> $a = x \pm r \cdot \cos(\theta)$, $b = y \pm r \cdot \sin(\theta)$, for $\theta \in [0°, 360°)$

Each edge pixel votes for all (a, b) on a cone in 3D parameter space at fixed r.

If radius is known: 2D accumulator A[a, b] sufficient — each pixel votes for a circle of radius r centred at (a, b).

Example from slides: Edge pixel at (230, 300), r = 50:

- θ = 0°: (a, b) = (280, 300) or (180, 300)
- θ = 90°: (a, b) = (230, 350) or (230, 250)

[Part A | Q37 | 5 Marks] What is generalized Hough Transform (GHT)?

Answer: The **Generalized Hough Transform (GHT)** extends HT to detect **arbitrary shapes** (not just lines, circles, ellipses) using a look-up table (R-table).

Key idea: Record the spatial relationship between each boundary point and the shape's reference point (centroid) in an R-table, indexed by gradient angle.

Training phase:

- For each boundary pixel of the template shape:
  - Compute gradient angle θ.
  - Compute vector (Δx, Δy) from this pixel to the centroid.
  - Store: R-table[θ] = (Δx, Δy).

Detection phase:

- For each edge pixel (x,y) in the input image:
  - Compute gradient angle θ.
  - Look up R-table[θ] → get (Δx, Δy).
  - Vote for centroid: accumulator[x−Δx, y−Δy]++.
- Peak in accumulator = shape location.

Advantages vs Standard HT: Works for any shape; no closed-form equation needed.

[Part A | Q38 | 5 Marks] What is spatial matched filtering?

Answer: Spatial matched filtering is a **template-based detection technique** where a reference template (filter) is cross-correlated with the input image to locate regions where the image best matches the template.

Formula:

> $C(x, y) = \Sigma_u \Sigma_v T(u, v) \cdot I(x+u, y+v)$

where T = template, I = image, C = correlation map.

Peaks in C(x,y) indicate locations where the image best matches the template.

Optimal matched filter: For detecting a known signal s(x,y) in Gaussian noise, the matched filter h(x,y) = s(-x,-y) maximizes the signal-to-noise ratio at the target location.

Relation to Hough Transform: Both are voting/correlation-based; HT uses parametric shape equations; matched filtering uses a pixel-level template.

Applications: Template-based object detection, logo/face detection, pattern recognition in satellite imagery.

[Part A | Q39 | 5 Marks] Define vanishing point.

Answer: A **vanishing point** is a point in a perspective (projective) image where **parallel 3D lines appear to converge** due to perspective projection onto the 2D image plane.

Example: Parallel railroad tracks appear to meet at a point on the horizon — the vanishing point.

Mathematical basis: In perspective projection, a family of 3D parallel lines with direction vector d all project to lines in the image that pass through a single 2D point (the vanishing point).

Detection using Hough Transform:

1. Apply Canny edge detection.
2. Use HT to detect lines (ρ, θ) in the edge image.
3. Find the intersection point of detected lines — this is the vanishing point.
4. Robust estimation using RANSAC.

Applications:

- Road lane detection and autonomous driving (vanishing point = road direction).

- Architectural photography (verify/correct perspective distortion).
- 3D scene reconstruction (camera calibration, depth estimation).
- Robot navigation (heading direction estimation).

[Part A | Q40 | 5 Marks] What is hole detection in binary images?

Answer: Hole detection identifies **enclosed background regions** (connected groups of 0-pixels completely surrounded by foreground 1-pixels) in a binary image.

Algorithm:

1. **Invert** the binary image (0→1, 1→0).
2. Apply **8-connected CCL** to the inverted image (background becomes foreground).
3. One large CC will include the outer boundary of the image — this is the **exterior background**.
4. All other CCs in the inverted image are **holes** (enclosed background regions).
5. Label holes separately from exterior background.

Alternative: Flood-fill from image border pixels → all reachable background pixels belong to exterior; remaining background pixels are holes.

Applications:

- Quality control (detecting holes in manufactured parts — e.g., drill holes in metal plates).
- Document analysis (letter 'O' has one hole; 'B' has two holes — topology-based character recognition).
- Shape analysis (Euler number = #objects − #holes).
- Medical imaging (detecting cavities, ventricles).

[Part B | B13 | 15 Marks] Derive the Hough Transform equation for line detection.

Answer:

Problem: Given a set of edge points in a 2D image, detect all straight lines passing through subsets of these points.

Step 1: Line Parameterization

A line in Cartesian form: $y = mx + b$ has parameters (m, b). However, vertical lines have $m \to \infty$ → unbounded parameter space.

Solution — Polar (Normal) Form:

Consider a line in 2D. The perpendicular from the origin to the line has:

- Length = ρ (perpendicular distance, signed)
- Angle = θ (angle of normal with x-axis)

Derivation: The line passes through point (x_0, y_0) on the perpendicular, which is at distance ρ from origin at angle θ:

- $x_0 = \rho\cdot\cos(\theta)$, $y_0 = \rho\cdot\sin(\theta)$

The line is perpendicular to the direction (cos θ, sin θ), so its equation is:

> $(x - x_0)\cdot\cos(\theta) + (y - y_0)\cdot\sin(\theta) = 0$

Expanding:

> $x\cdot\cos(\theta) - x_0\cdot\cos^2(\theta) + y\cdot\sin(\theta) - y_0\cdot\sin^2(\theta) = 0$ $x\cdot\cos(\theta) + y\cdot\sin(\theta) = x_0\cdot\cos^2(\theta) + y_0\cdot\sin^2(\theta)$ $x\cdot\cos(\theta) + y\cdot\sin(\theta) = \rho\cdot\cos^2(\theta) + \rho\cdot\sin^2(\theta) = \rho$

Final Hough Transform Equation:

> **$\rho = x\cdot\cos(\theta) + y\cdot\sin(\theta)$**

where: $\rho \in [-d_max, d_max]$, $\theta \in [0°, 180°)$, $d_max = \sqrt{(W^2+H^2)}$.

Step 2: Hough Space Mapping

Each image point (x_i, y_i) maps to a **sinusoidal curve** in (ρ, θ) Hough space:

> $\rho = x_i\cdot\cos(\theta) + y_i\cdot\sin(\theta)$

For multiple collinear points (x_1,y_1), (x_2,y_2), (x_3,y_3):

- Each maps to a sinusoid in (ρ,θ) space.
- All sinusoids of collinear points *intersect at a single (ρ,θ)** — the parameters of their common line.

Step 3: Accumulator Voting Algorithm

```
Initialize A[ρ][θ] = 0

For each edge pixel (x, y):
    For θ = 0 to 180° step Δθ:
        ρ = x·cos(θ) + y·sin(θ)
        Discretize ρ to nearest bin
        A[ρ][θ] += 1

Find peaks in A[ρ][θ] exceeding threshold T
Each peak (ρ*, θ*) defines a detected line
```

Step 4: Numerical Example

Points: (1,2), (2,3), (3,4) — check if collinear using HT.

For point (1,2): c = y − mx → c = 2 − m(1)

- m=0: c=2 → (m,c) = (0,2)
- m=1: c=1 → (m,c) = (1,1)
- m=1.5: c=0.5

For point (2,3): c = 3 − 2m

- m=0: c=3; m=1: c=1 → (1,1)
- m=1.5: c=0 → (1.5,0)

For point (3,4): c = 4 − 3m

- m=1: c=1 → **(1,1)** ← all three agree! Line: y = x + 1.

All three sinusoids intersect at (m,c) = (1,1) → line y = x + 1. Points are collinear.

Step 5: Peak Detection & Line Extraction

After accumulation:

1. Apply non-maximum suppression on A[ρ][θ].
2. Extract peaks above threshold T.
3. Convert each (ρ*, θ*) back to line: draw all pixels satisfying ρ* = x·cos(θ*) + y·sin(θ*).

[Part B | B14 | 15 Marks] Explain foot-of-normal method for line localization.

Answer:

Definition: The foot-of-normal method parameterizes lines by the perpendicular (normal) from the origin to the line — using distance ρ and angle θ.

Geometric Construction:

```
y
|         Line L
|        /
|   ρ  /  (foot F)
|----F
|  θ/
| /
0------------ x
```

- O = origin (0,0)
- F = foot of perpendicular from O to line L
- ρ = |OF| = perpendicular distance
- θ = angle OF makes with x-axis

Line Equation Derivation:

Vector OF = (ρ·cos θ, ρ·sin θ)

Line L is perpendicular to OF, so its direction vector = (−sin θ, cos θ).

Line passes through F: parametrically $P = F + t(-\sin\theta, \cos\theta)$:

- $x = \rho\cdot\cos\theta - t\cdot\sin\theta$
- $y = \rho\cdot\sin\theta + t\cdot\cos\theta$

Eliminate t: multiply first by $\cos\theta$, second by $\sin\theta$:

> $x\cdot\cos\theta + y\cdot\sin\theta = \rho\cdot\cos^2\theta + \rho\cdot\sin^2\theta = \boldsymbol{\rho}$

Foot-of-Normal Equation: $\rho = x\cdot\cos\theta + y\cdot\sin\theta$

Why "Foot-of-Normal":

The foot $F = (\rho\cos\theta, \rho\sin\theta)$ is the point on the line **closest to the origin** (the foot of the perpendicular). The method is named after this geometric construction.

Localization Algorithm:

Step 1: Apply Canny edge detection to get binary edge map.

Step 2: For each edge pixel (x, y):

- For $\theta \in \{0°, 1°, 2°, ..., 179°\}$:
  - Compute $\rho = x\cdot\cos(\theta) + y\cdot\sin(\theta)$
  - Increment accumulator $A[\text{round}(\rho)][\theta]$

Step 3: Find peaks in $A[\rho][\theta]$:

- Sort by vote count.
- Apply non-maximum suppression in accumulator.

Step 4: Each peak (ρ^*, θ^*) defines a line:

- The line passes through $F = (\rho^*\cos\theta^*, \rho^*\sin\theta^*)$.
- Direction: $(-\sin\theta^*, \cos\theta^*)$.

Step 5: Reconstruct line endpoints:

- Find edge pixels closest to the detected line (within distance threshold).
- Use their extent to determine start and end points of line segment.

Advantages of Foot-of-Normal:

Property	Slope-Intercept (m,b)	Foot-of-Normal (ρ,θ)
Vertical lines	Fails ($m\to\infty$)	Handled ($\theta=90°$)
Parameter range	Unbounded m	Bounded $\rho \in [-d,d]$
Accumulator size	Infinite	Finite
Geometric intuition	Low	High (normal to line)

[Part B | B15 | 15 Marks] Discuss limitations of Hough Transform and speed problem.

Answer:

Standard Hough Transform for Lines:

Strengths:

- Robust to noise and missing edge pixels.
- Detects lines even in high-noise images.
- Handles multiple lines simultaneously.
- Works with occluded lines.

LIMITATIONS:

1. Computational Complexity (Speed Problem):

- For each edge pixel, compute ρ for all θ values (e.g., 360 steps).
- N edge pixels × 360 θ values = N × 360 operations.
- For a 512×512 image with many edge pixels: millions of accumulator updates.
- For circles (3D accumulator): O(N × 360 × R_max) — much worse.
- **Solution**: Randomized Hough Transform (RHT) — randomly sample pairs/triples of edge points instead of all.

2. Memory Requirements:

- Line: 2D accumulator (manageable).
- Circle (unknown r): 3D accumulator A[a][b][r] — O(W×H×R) cells.
- Ellipse: 5-parameter space — impractical for brute force.
- **Solution**: Use gradient information to reduce dimensions (gradient-based CHT uses 2D).

3. Quantization Effects:

- Coarse (ρ, θ) quantization: nearby lines merged → imprecise detection.
- Fine quantization: large accumulator, slow.
- **Solution**: Hierarchical HT — coarse-to-fine search; subpixel refinement at peaks.

4. Parameter Space vs. Detected Shape:

- A peak in HT space only gives line parameters — not start/end points of a line segment.
- Need post-processing to determine line extent from edge pixels.
- **Solution**: Probabilistic HT — tracks which pixels voted for each peak.

5. Sensitivity to Threshold:

- Too low threshold: many false positive lines from noise.
- Too high threshold: true lines with few edge pixels missed.
- **Solution**: Adaptive thresholding based on local accumulator statistics.

6. Only Parametric Shapes:

- Standard HT restricted to lines, circles, ellipses (shapes with closed-form equations).
- **Solution**: Generalized Hough Transform (GHT) for arbitrary shapes — but uses R-table (memory-intensive).

7. Parallel Lines / Clutter:

- Multiple parallel lines (e.g., road lanes) may create multiple closely spaced peaks.
- **Solution**: Peak clustering in accumulator space; non-maximum suppression.

Speed Improvement Techniques:

Technique	Description	Speedup
Gradient-Based HT	Only vote in gradient direction ±θ window	10-100×
Randomized HT (RHT)	Randomly sample point pairs/triples	Significant
Hierarchical HT	Coarse-to-fine accumulator	5-10×
RANSAC (alternative)	Random sampling instead of voting	Very fast
Edge-only voting	Skip non-edge pixels	Proportional to sparsity

---

[Part B | B16 | 15 Marks] Explain circle detection using Hough Transform.

Answer:

Circle Equation (Parametric Form):

> $(x - a)^2 + (y - b)^2 = r^2$

Parameters: centre (a, b) and radius r → 3D parameter space.

Case 1: Known Radius r

2D accumulator A[a][b]:

- For each edge pixel (x, y):
  - For θ = 0° to 360°:
    - a = x − r·cos(θ)
    - b = y − r·sin(θ)
    - A[a][b]++

Peaks in A[a][b] give circle centres.

Case 2: Unknown Radius r

3D accumulator A[a][b][r]:

- For each edge pixel (x, y):
  - For each r in [r_min, r_max]:

    - For θ = 0° to 360°:
        - a = x − r·cos(θ); b = y − r·sin(θ)
        - A[a][b][r]++

Peaks in 3D accumulator give (a*, b*, r*).

Gradient-Based Optimization:

Using edge direction θ_gradient, the centre must lie along the gradient direction:

> a = x − r·cos(θ_gradient) b = y − r·sin(θ_gradient)

This reduces the 3D voting to essentially 2D + r sweep, cutting computation significantly.

Full Circular HT Algorithm:

```
Step 1: Apply Canny edge detection → binary edge map E

Step 2: Initialize A[a][b][r] = 0

Step 3: For each edge pixel (x,y) in E:
    For r = r_min to r_max:
        For θ = 0 to 360° step Δθ:
            a = x − r·cos(θ)
            b = y − r·sin(θ)
            if (a,b,r) in valid range:
                A[a][b][r]++

Step 4: Find peaks in A[a][b][r] > threshold T

Step 5: Each peak (a*, b*, r*) → detected circle
```

Numerical Example (from slides):

Image: 512×512. Circle: centre (200,300), radius r=50. Edge pixel detected at (230, 300).

For θ=0°: a = 230±50·1 = 280 or 180; b = 300±0 = 300 For θ=90°: a = 230±0 = 230; b = 300±50 = 350 or 250

Votes cast at: (280,300), (180,300), (230,350), (230,250).

When all edge pixels on the circle vote, accumulator cell (200,300,50) gets maximum votes → detected circle.

Applications:

- Iris/pupil detection (human eye circular boundary).
- Coin detection and counting.

- Cell detection in microscopy.
- Ball detection in sports vision.
- Quality control (hole detection in metal plates).

[Part B | B17 | 15 Marks] Explain ellipse detection using Generalized Hough Transform.

Answer:

Why GHT for Ellipses:

- An ellipse has 5 parameters: (a, b, cx, cy, θ_orient) → 5D parameter space is impractical for standard HT.
- GHT avoids closed-form equations by using a template-based R-table approach.

GHT Overview: The Generalized Hough Transform detects arbitrary shapes by building a look-up table (R-table) from a reference template and using it to vote for object positions in a new image.

Phase 1: Training (Building the R-table)

Given template ellipse image:

1. Apply edge detection to template → get boundary pixels.
2. Select a reference point (centroid): (x_ref, y_ref).
3. For each boundary pixel (x_i, y_i): a. Compute gradient angle θ_i at (x_i, y_i). b. Compute offset vector: (Δx_i, Δy_i) = (x_ref − x_i, y_ref − y_i). c. Store: R-table[θ_i] ← append (Δx_i, Δy_i).

R-table structure:

```
θ = 0°  → [(Δx₁, Δy₁), (Δx₂, Δy₂), ...]
θ = 10° → [(Δx₃, Δy₃), ...]
...
θ = 350° → [(Δxₙ, Δyₙ)]
```

Phase 2: Detection

Given input image with possible ellipses:

1. Apply edge detection → edge map.
2. For each edge pixel (x, y): a. Compute gradient angle θ at (x, y). b. Look up R-table[θ] → list of offsets (Δx, Δy). c. For each (Δx, Δy) in R-table[θ]:
   - Candidate centroid: (a, b) = (x + Δx, y + Δy)
   - Vote: Accumulator[a][b]++
3. Find peaks in Accumulator[a][b].
4. Each peak = detected ellipse centroid location.

Worked Example (from slides):

Template ellipse with 3 edge points:

θ	Offset (Δx, Δy)
45°	(−5, −10)
90°	(5, 0)
135°	(0, 10)

Detected edge at (50, 60):

- θ=45°: center = (50−(−5), 60−(−10)) = (55, 70) → vote.
- θ=90°: center = (50−5, 60−0) = (45, 60) → vote.
- θ=135°: center = (50−0, 60−10) = (50, 50) → vote.

When multiple edge pixels vote, peak accumulates at the true centroid.

Extensions for Ellipses:

For scale/rotation-invariant detection, extend accumulator to include scale s and rotation ϕ:

- Accumulator[a][b][s][ϕ] — 4D.
- For each (Δx, Δy) in R-table: vote for (x + s·R(ϕ)·Δx, y + s·R(ϕ)·Δy).

Advantages of GHT:

- Works for any shape (ellipse, triangle, heart, etc.).
- Uses same voting framework as standard HT.
- R-table is computed once, reused for all detections.

Limitations:

- R-table must be rebuilt for each new shape.
- Large memory for R-table with many gradient angles.
- Sensitive to rotation/scale if not accounted for (requires higher-dim accumulator).

[Part B | B18 | 15 Marks] Compare Hough Transform and RANSAC for line detection.

Answer:

Hough Transform (HT) for Line Detection:

Method: Each edge pixel votes for all lines passing through it in parameter space. Peaks in accumulator = detected lines.

Algorithm:

1. Edge detection.
2. For each edge pixel → vote in (ρ,θ) space.
3. Find accumulator peaks.
4. Each peak = one line.

Complexity: O(N×K) where N = edge pixels, K = θ quantization steps.

RANSAC for Line Detection:

Method: Randomly sample minimal point sets, fit lines, count inliers, keep best.

Algorithm:

1. Randomly pick 2 points from edge set.
2. Fit line y = mx + c through them.
3. Count inliers: points within distance ε of line.
4. If inlier count > best, save model.
5. Repeat $N = \log(1-P)/\log(1-w^2)$ iterations.
6. Refit using all inliers.

Complexity: O(N_iter × N) where N_iter = number of RANSAC iterations (typically 100–1000).

Detailed Comparison:

Property	Hough Transform	RANSAC
Approach	Dense voting in parameter space	Random sampling + consensus
Output	All lines simultaneously	One line per run (best model)
Multi-line detection	Natural (multiple peaks)	Requires iterative RANSAC with inlier removal
Noise robustness	Good (all pixels vote)	Very good (explicitly handles outliers)
Outlier handling	Implicit (noise spreads votes)	Explicit (outliers excluded from model)
Parameter choice	Δρ, Δθ (accumulator resolution)	ε (inlier threshold), P (confidence)
Computation	Slower for many edge pixels	Faster when inlier ratio is high
Accuracy	Resolution limited by quantization	Subpixel-accurate (analytical fit)
Memory	O(ρ_bins × θ_bins)	O(N) only
Arbitrary shapes	Requires GHT extension	Easy — just change the model fitting step
Missing edges	Tolerant	Requires minimum inlier count
Implementation	Moderate	Simple

When to Use Each:

Use HT when:

- Multiple shapes must be detected simultaneously.
- Computation is not the bottleneck.
- Parameter space is low-dimensional (≤ 3D).
- Images are moderately noisy with many edge pixels.

Use RANSAC when:

- High outlier ratio (> 50% outliers).
- Need subpixel accuracy.
- Multiple models can be found by iterating with inlier removal.
- Fast runtime is required.
- Complex models (homography, fundamental matrix — 8 parameters).

Combined Approach: In practice, both are often used together:

1. HT gives rough initial line hypotheses.
2. RANSAC refines each hypothesis using pixel evidence.

[Part B | B19 | 15 Marks] Explain steps involved in human iris localization using circular Hough Transform.

Answer:

Motivation: The human iris is roughly circular. Iris localization finds the inner (pupil-iris) and outer (iris-sclera) boundaries in an eye image. CHT is ideal because it detects circles robustly even with partial occlusion (eyelids, eyelashes).

System Overview:

```
Eye Image → Pre-processing → Edge Detection
→ Circular HT → Peak Detection
→ Inner Circle (pupil) → Outer Circle (iris)
→ Iris Region Extracted
```

Step 1: Image Acquisition & Pre-processing

- Capture near-infrared (NIR) eye image (iris contrast is higher in NIR than visible light).
- **Grayscale conversion** if colour image.
- **Histogram equalization**: enhance contrast between iris, pupil, and sclera.
- **Gaussian smoothing**: $G_\sigma * I$ to reduce noise before edge detection.

Step 2: Edge Detection

- Apply **Canny edge detector** to produce binary edge map.
- Parameters: σ ≈ 1–2 (moderate smoothing), T_high, T_low set empirically.
- The Canny detector produces strong, thin edges at iris/pupil/sclera boundaries.

Step 3: Circular Hough Transform — Pupil (Inner Circle)

Parameters: centre (a, b), radius r ∈ [r_min_p, r_max_p] (pupil size range ≈ 20–80 pixels).

Voting:

```
Initialize A_inner[a][b][r] = 0
For each edge pixel (x, y):
    For r = r_min_p to r_max_p:
        For θ = 0 to 360°:
            a = x − r·cos(θ)
            b = y − r·sin(θ)
            A_inner[a][b][r]++
```

Peak Detection: Find (a_p, b_p, r_p) = argmax A_inner.

- This is the **pupil circle**.

Step 4: Circular Hough Transform — Iris (Outer Circle)

- Use detected pupil centre (a_p, b_p) as a constraint.
- Search for outer circle concentric or near-concentric with pupil.
- Radius range: r ∈ [r_min_iris, r_max_iris] (iris ≈ 100–150 pixels radius).
- Optionally mask eyelid regions before voting (upper/lower eyelid regions suppressed using parabolic masking).

Voting (similar to above with r_iris range). **Peak Detection**: Find (a_i, b_i, r_i) = iris circle.

Step 5: Eyelid/Eyelash Masking

- Eyelids create curved non-circular edges that generate false votes.
- Fit parabola to upper/lower eyelid using another HT.
- Exclude pixels in eyelid parabola regions from iris CHT voting.

Step 6: Iris Region Extraction

- The iris region = annular region between inner circle (pupil, radius r_p) and outer circle (iris, radius r_i):
  - Include pixels where $r_p < \sqrt{(x-a_i)^2+(y-b_i)^2} < r_i$.
- This annular region contains the iris texture used for biometric recognition.

Step 7: Iris Normalization (Daugman's Rubber Sheet Model)

- Unwrap the annular iris region into a rectangular image for consistent feature extraction.
- Map (r, θ) in iris → (x, y) in rectangular template.

Post-processing:

- Extract features from normalized iris (Gabor filter → phase quantization → IrisCode).
- Match IrisCode using Hamming distance for biometric identification.

Key Parameters:

- Pupil radius range: 20–80 pixels (typical for 640×480 images).
- Iris radius range: 80–150 pixels.
- Canny σ: 1.5–2.0.
- CHT Δθ: 1° (360 votes per pixel per radius).

[Part B | B20 | 15 Marks] Explain vanishing point detection in perspective images.

Answer:

Definition: A vanishing point (VP) is the image location where parallel 3D lines converge in perspective projection. Detecting VPs helps understand scene geometry and 3D structure.

Why Vanishing Points Occur:

In perspective projection, a set of parallel 3D lines with direction **d = (d_x, d_y, d_z)** project to image lines that all pass through the same 2D point:

> VP = (f·d_x/d_z, f·d_y/d_z) (in image coordinates, f = focal length)

Example: Horizontal road lanes → parallel → VP at horizon. Building vertical edges → VP above image.

Types of Vanishing Points:

- **Horizontal VP**: Due to horizontal parallels (road, floor tiles).
- **Vertical VP**: Due to vertical structures (buildings, poles).
- **Diagonal VP**: Due to diagonal lines in the scene.
- **Orthogonal scenes** have 3 mutually perpendicular VPs (Manhattan World assumption).

Method 1: Hough Transform Based VP Detection

Steps:

1. **Pre-processing**: Convert to grayscale, apply Gaussian smoothing.
2. **Edge Detection**: Apply Canny edge detector.
3. **Line Detection**: Apply Hough Transform (ρ,θ space) to detect dominant lines in the edge image.
4. **Line Intersection**: For each pair of detected lines L_1, L_2:
    - Compute intersection point (x,y) = solve:

    - $\rho_1 = x \cdot \cos(\theta_1) + y \cdot \sin(\theta_1)$
    - $\rho_2 = x \cdot \cos(\theta_2) + y \cdot \sin(\theta_2)$
5. **Voting for VP**: Each line intersection votes for a VP candidate.
6. **VP as Mode**: The most common intersection region = vanishing point.

Limitation: Pairwise intersections are $O(n^2)$ for n lines. Many spurious intersections.

Method 2: RANSAC-Based VP Detection

1. Detect line segments (LSD — Line Segment Detector or Hough).
2. Randomly select 2 line segments.
3. Compute their intersection (VP candidate).
4. Count **inlier lines**: line segments that pass within distance ε of the candidate VP.
5. Keep VP candidate with most inliers.
6. Repeat for N iterations.
7. Refit VP using all inlier lines' intersection (weighted average).

Advantage: Robust to outlier lines (textures, reflections).

Method 3: Accumulator Space (Gaussian Sphere)

Map each line as a point on the Gaussian sphere (dual space). Lines through a common VP map to a great circle. VPs become intersections of great circles → detected as density maxima on sphere.

Algorithm (Hough-based, detailed):

```
1. I_gray = grayscale(I_input)
2. I_smooth = G_σ ∗ I_gray
3. Edges = Canny(I_smooth)
4. Lines = HoughTransform(Edges) → list of (ρᵢ, θᵢ)
5. Intersections = []
   For each pair (Lᵢ, Lⱼ):
       Solve: [cos θᵢ  sin θᵢ][x]   [ρᵢ]
              [cos θⱼ  sin θⱼ][y] = [ρⱼ]
       (x,y) = intersection
       Intersections.append((x,y))
6. Cluster Intersections using mean-shift or k-means
7. VP = cluster centre with maximum density
```

Post-processing:

- Multiple VPs can be detected by separating lines by direction group (θ ≈ 0°, 45°, 90°) before finding VPs.
- Road VP: lines with θ ≈ 45°–135° (angled lane markings).
- Building VP: lines with θ ≈ 0° (horizontal) and θ ≈ 90° (vertical).

Applications:

1. **Autonomous Driving**: VP gives driving direction; lane departure warning.
2. **Camera Calibration**: 3 orthogonal VPs give camera intrinsic parameters.
3. **3D Reconstruction**: VPs define coordinate axes of the scene.
4. **Image Rectification**: Correct perspective distortion using VP.
5. **Augmented Reality**: Align virtual objects with scene geometry.

End of CVT Unit 2 Answer Sheet — 40 Part A + 20 Part B answers, grouped by 7 topics.